

ELCO-Dealer
Guía software
Electrónica de Consumo

JAVIER CARRETERO,
CARLOS GONZÁLEZ,
ITZIAR OTAÑO,
ÁLVARO POVEDA,
CRISTINA RODRÍGUEZ,
JUAN SALGADO

Universidad Politécnica de Madrid
12 de mayo de 2026



Índice

1. Introducción	2
2. Objetivo y alcance	2
3. Arquitectura del software embebido	2
3.1. Criterios de diseño	2
3.2. Estructura del repositorio	2
3.3. Mapa de pines y elementos controlados	3
3.4. Máquina de estados finitos	4
3.5. Gestión de eventos de usuario	4
3.6. Control de sensores	4
3.7. Programación de los motores	5
3.8. Timeouts y recuperación de errores	5
3.9. Modo de diagnóstico	5
3.10. Compilación y carga	6
3.11. Parámetros de calibración	6
4. Página web	7
4.1. Arquitectura y Contenidos de la Plataforma	7
4.2. Estrategia de Captación: Wishlist y Reserva	8
5. Conclusión	10



1. Introducción

Esta guía documenta el software desarrollado para el prototipo **ELCO-Dealer**, realizado como parte de la asignatura **Electrónica de Consumo** del Grado en Ingeniería de Tecnologías y Servicios de Telecomunicación de la Universidad Politécnica de Madrid.

2. Objetivo y alcance

La guía tiene como objetivo facilitar el mantenimiento y la ampliación del software del **ELCO-Dealer**.

El software se ha desarrollado en **C++ para Arduino** y se ejecuta sobre una **Arduino Mega 2560 R3**.

3. Arquitectura del software embebido

3.1. Criterios de diseño

Los criterios principales han sido los siguientes:

- **Determinismo:** el sistema debe responder siempre de la misma forma ante la misma secuencia de eventos.
- **No bloqueo:** el código evita esperas con `delay()` y sus actualizaciones periódicas se basan en `millis()`.
- **Separación por capas:** la FSM decide qué debe ocurrir, pero no manipula pines directamente.
- **Seguridad de operación:** los movimientos de giro y empuje incorporan *timeouts* para evitar que un motor quede funcionando indefinidamente ante un fallo de sensor.
- **Calibración centralizada:** pines, umbrales, tiempos y velocidades PWM se agrupan en `src/config.h`.

3.2. Estructura del repositorio

El punto de entrada del programa es mínimo. El archivo `card_dealer.ino` únicamente inicializa la máquina de estados en `setup()` y la actualiza de forma continua en `loop()`. El comportamiento real se delega en los módulos de `src/`.



Archivo	Responsabilidad
card_dealer.ino	Llama a fsm_init() y fsm_update().
src/config.h	Define pines, límites de jugadores, tiempos de seguridad, PWM, polaridad de sensores y umbrales de la botonera.
src/config_checks.h	Verifica que no existan duplicados de pines o algunas otras complicaciones.
src/fsm.*	Implementa la lógica del sistema mediante una máquina de estados finitos. Decide cuándo girar, empujar cartas, volver a HOME o mostrar que ha habido un error.
src/hardware.*	Centraliza el acceso a motores, drivers, el sensor Hall HOME y los sensores de paso de cartas.
src/ui.*	Gestiona la pantalla y los botones.
src/diagnostics.*	Permite probar sensores y motores de forma independiente.
src/debug.*	Encapsula los mensajes por Serial.

Cuadro 1: Organización del software del ELCO-Dealer

3.3. Mapa de pines y elementos controlados

El mapa de pines se encuentra en Config.

Elemento	Pines configurados
LCD 16x2	D4–D7 en 4, 5, 6 y 7; RS en 8; EN en 9; retroiluminación en 10.
Botonera LCD Keypad	Entrada analógica A0.
L298N 1	PWM en 11 y 12; dirección en 22, 23, 24 y 25.
L298N 2	PWM en 2 y 3; dirección en 26, 27, 28 y 29.
Sensor de carta activo	CARD_SENSOR_1, CARD_SENSOR_2 y CARD_SENSOR_3 en los pines 30, 31 y 32 respectivamente.
Sensor HOME	Sensor Hall en el pin 33.

Cuadro 2: Resumen del mapa de pines de src/config.h



3.4. Máquina de estados finitos

Estado	Función dentro del sistema
OFF	Estado inicial. Motores están detenidos y la pantalla indica que se mantenga pulsado SELECT.
INIT	Estado posterior al encendido. Reinicia contadores e inicia el menú de configuración.
SET_PLAYERS	Selección del número de jugadores, con límite máximo de 6.
SET_CARDS	Selección del número de cartas por jugador.
AUTO_INIT	Preparación del reparto automático.
ROTATE_AUTO	Gira la plataforma hasta la posición del jugador actual.
PUSH_AUTO	Acciona el motor de reparto hasta detectar una carta mediante el sensor de paso de carta.
RETURN_HOME_AUTO	Devuelve el sistema a la posición HOME tras completar la ronda.
IDLE_HOME	Estado de reposo. Permite iniciar otra ronda o repartir una carta manual.
ROTATE_ONE	Gira hacia el siguiente jugador en modo manual.
PUSH_ONE	Reparte una única carta.
RETURN_HOME_MANUAL	Vuelve a HOME tras el reparto manual.
ERROR	Estado tras un fallo. Los motores se detienen.

Cuadro 3: Estados principales implementados en `src/fsm.cpp`

A cada jugador se le asigna un slot, este slot depende del número de jugadores:

$$\text{slot objetivo} = \left\lfloor \frac{\text{índice de jugador} \cdot 6}{\text{número de jugadores}} \right\rfloor$$

Gracias a esto la distancia entre los jugadores es siempre uniforme.

3.5. Gestión de eventos de usuario

- **PWR:** asociado al botón SELECT. Una pulsación corta confirma o avanza; una pulsación larga enciende o apaga el sistema.
- **ADD:** asociado al botón RIGHT. Incrementa jugadores, cartas o lanza una carta manualmente (dependiendo del estado en el que nos encontremos).

3.6. Control de sensores

Los sensores digitales se gestionan desde `hardware.cpp`. Cada sensor mantiene su propio estado, último valor leído, instante del último cambio, instante del último evento y banderas de flanco ascendente o descendente.

El sensor Hall HOME utiliza:

- lectura digital con nivel activo en `HOME_SENSOR_ACTIVE_LEVEL`;



- antirrebote mediante HALL_DEBOUNCE_MS;
- tiempo muerto mediante HALL_DEAD_TIME_MS;
- rearme en la FSM mediante HALL_REARM_MS.

El sensor de paso de carta sigue la misma filosofía, pero con tiempos definidos como CARD_SENSOR_DEBOUNCE_MS, CARD_SENSOR_DEAD_TIME_MS y CARD_SENSOR_REARM_MS. De esta forma se evita contar varias veces una misma carta.

3.7. Programación de los motores

Todos los motores se controlan desde la capa Hardware.

Cada motor se representa mediante:

- un pin PWM conectado al enable del L298N;
- dos pines de dirección;
- una velocidad PWM actual;
- una dirección lógica.

El motor de rotación se controla como MotorId::ROTATION y los motores de reparto como DEAL_1, DEAL_2 y DEAL_3.

La velocidad inicial de giro se define en ROTATION_PWM y la velocidad de reparto en DEAL_PWM.

3.8. Timeouts y recuperación de errores

Las acciones con movimiento tienen un tiempo máximo de ejecución:

Parámetro	Uso
ROTATE_TIMEOUT_MS	Tiempo máximo para alcanzar un slot durante el giro.
PUSH_TIMEOUT_MS	Tiempo máximo para detectar una carta.
RETURN_HOME_TIMEOUT_MS	Tiempo máximo para detectar HOME durante el retorno.

Cuadro 4: Timeouts principales de seguridad

Si se supera alguno de estos límites, la FSM entra en ERROR, detiene todos los motores y muestra el tipo de fallo en la pantalla.

3.9. Modo de diagnóstico

El modo de diagnóstico permite validar el prototipo antes de ejecutar la FSM completa.

En este modo, el usuario navega por páginas de prueba usando la pantalla LCD:



- **PWR corto:** avanzar a la siguiente página de diagnóstico.
- **ADD corto:** activar o detener el motor mostrado en pantalla.
- **PWR largo:** vuelta a la página HOME.

Se recomienda comprobar primero el sensor HOME, después el sensor de paso de carta y finalmente cada motor.

3.10. Compilación y carga

El proyecto está preparado para compilarse con Arduino CLI usando la placa **Arduino Mega**. En modo normal:

```
arduino-cli compile --fqbn arduino:avr:mega .
```

Para compilar el modo de diagnóstico:

```
arduino-cli compile --fqbn arduino:avr:mega '
  --build-property compiler.cpp.extra_flags=-DCARD_DEALER_DIAGNOSTIC_MODE=true .
```

La carga en placa debe realizarse indicando el puerto serie real detectado en el equipo:

```
arduino-cli upload -p COMx --fqbn arduino:avr:mega .
```

3.11. Parámetros de calibración

La calibración final debe hacerse con el prototipo físico montado.

Grupo	Parámetros a revisar
Botonera	Umbrales KEYPAD_RIGHT_MAX, KEYPAD_UP_MAX, KEYPAD_DOWN_MAX, KEYPAD_LEFT_MAX y KEYPAD_SELECT_MAX.
Sensores	Polaridad CARD_SENSOR_ACTIVE_LEVEL y HOME_SENSOR_ACTIVE_LEVEL.
Filtrado	Tiempos de antirrebote, tiempo muerto y rearme para Hall y sensor de carta.
Motores	ROTATION_PWM y DEAL_PWM.
Seguridad	ROTATE_TIMEOUT_MS, PUSH_TIMEOUT_MS y RETURN_HOME_TIMEOUT_MS.

Cuadro 5: Parámetros software que requieren calibración física



4. Página web

Como parte de la comercialización del proyecto, se ha diseñado una plataforma en la dirección: <https://fantastic-douhua-aab9a0.netlify.app/>.

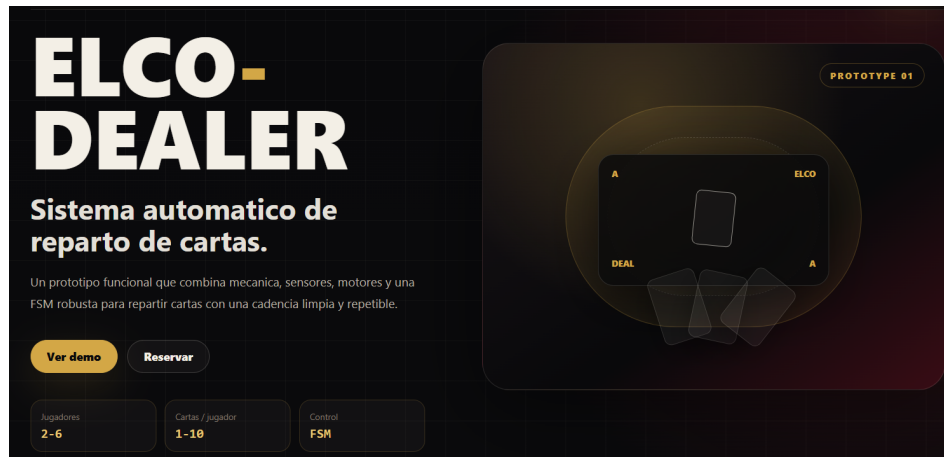


Figura 1: Interfaz principal de la web ELCO-Dealer.

4.1. Arquitectura y Contenidos de la Plataforma

- **Highlights y Demo:** Esta sección presenta la propuesta de valor del **ELCO-Dealer**. Incluye un video que muestra el funcionamiento real del prototipo, su capacidad de barajado automático y el modo de uso (ver Figura 1).
- **Simulador de Reparto:** Herramienta interactiva (Figura 2) que permite visualizar el proceso de distribución de cartas. Emula el comportamiento del software de la Arduino Mega 2560.
- **Engineering (Arquitectura de Software):** Explica el diseño orientado a la robustez (Figura 3), destacando el uso de una **Máquina de Estados Finitos (FSM)**. La web explica cómo el sistema evita bloqueos implementando prioridades de eventos y *timeouts*.
- **Hardware y Sensórica:** Bloque que describe como se integran los componentes clave (BOM). Se resalta el uso de los drivers **L298N** para el control de potencia [1], el sensor **A3144** para la detección de "home" los sensores infrarrojos **TCRT5000** para la detección del paso de las cartas.
- **User Interface (UI):** Detalla como funciona el sistema de configuración (el que permite al usuario definir el número de jugadores y de cartas por jugador).

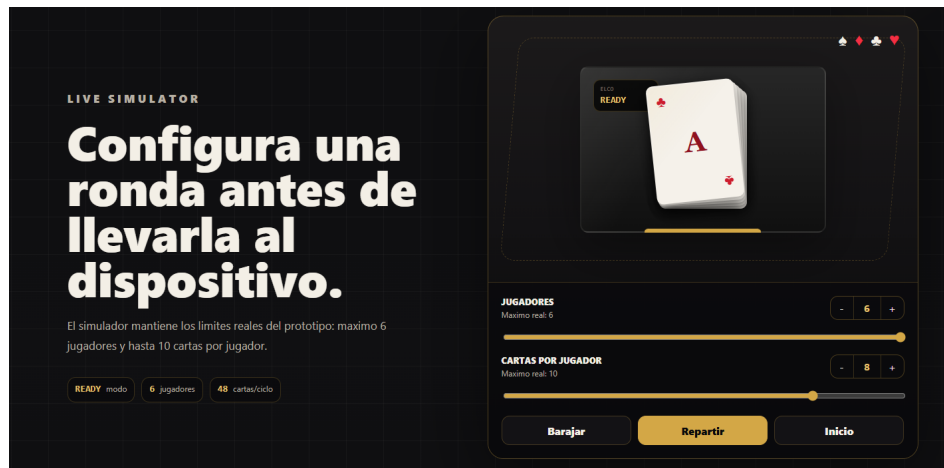


Figura 2: Simulador de barajado y reparto.



Figura 3: Información sobre la FSM, el hardware y la UI del prototipo.

4.2. Estrategia de Captación: Wishlist y Reserva

La web incluye una sección de reserva (Figura 4). Esta función permite mantener un registro de posibles clientes.



The image shows a web form for 'ELCO-DEALER Prototype Access'. On the left, under the heading 'RESERVE', is the text 'A polished concept, ready to be demonstrated.' followed by a paragraph in Spanish explaining that the product is a prototype and users can join a waitlist or watch a demo. Below this are two buttons: 'Join the Waitlist' and 'Watch Demo'. On the right, the form is titled 'Concept reservation' and lists three bullet points: 'University engineering prototype', 'Functional motion and sensing system', and 'Designed for future public demo assets'. A note states that public release is not available yet and that this is a waitlist-style interaction. The form includes input fields for 'NOMBRE' (Tu nombre) and 'EMAIL' (tu@email.com), a dropdown for 'INTERES' (Comprar una unidad), and a final button 'Unirme a wishlist'.

RESERVE

A polished concept, ready to be demonstrated.

ELCO-DEALER todavía no es un producto ecommerce público. Esta reserva presenta el prototipo como un lanzamiento creíble, manteniendo la acción honesta: registra interés, sigue la demo y acompaña el hito de ingeniería.

[Join the Waitlist](#) [Watch Demo](#)

ELCO-DEALER Prototype Access

Concept reservation

- University engineering prototype
- Functional motion and sensing system
- Designed for future public demo assets

Public release is not available yet. This is a waitlist-style interaction for presentation and product storytelling.

NOMBRE **EMAIL**

Tu nombre tu@email.com

INTERES

Comprar una unidad

[Unirme a wishlist](#)

Figura 4: Formulario de reserva y wishlist integrado en la plataforma web.



5. Conclusión

El software del **ELCO-Dealer** se ha planteado como una base para un prototipo físico sometido a posibles rebotes de sensores y pequeñas variaciones en el movimiento de cartas.

Como líneas de mejora se podrían registrar métricas de fallo durante el uso real, modificar el funcionamiento según el tipo de baraja y conectar la electrónica con una aplicación móvil .



Referencias

1. STMicroelectronics, *L298N Dual Full-Bridge Driver*, 2019, datasheet oficial. [Online]. Available: <https://www.st.com/resource/en/datasheet/l298.pdf>
2. Arduino, *Arduino Mega 2560 Rev3*, 2024, datasheet oficial. [Online]. Available: <https://docs.arduino.cc/hardware/mega-2560/>
3. STMicroelectronics, *L298N Dual Full-Bridge Driver*, 2019, datasheet oficial. [Online]. Available: <https://www.st.com/resource/en/datasheet/l298.pdf>
4. Adafruit Industries, *DC Gearbox Motor - "TT Motor 200RPM - 3 to 6VDC*, 2018. [Online]. Available: <https://www.adafruit.com/product/3777>
5. Adafruit, *Slip Ring 22mm Diameter, 6 Wires, 2A (SRC022A-6)*, 2020. [Online]. Available: <https://www.adafruit.com/product/736>
6. Vishay Semiconductors, *TCRT5000 Reflective Optical Sensor*, 2009. [Online]. Available: <https://www.vishay.com/docs/83760/tcrt5000.pdf>
7. Allegro MicroSystems, *A3144 Sensitive Hall-Effect Switch*, 2005. [Online]. Available: <https://www.alldatasheet.com/datasheet-pdf/pdf/55092/ALLEGRO/A3144.html>
8. DFRobot, *DFRobot LCD Keypad Shield (DFR0009)*, 2011. [Online]. Available: https://wiki.dfrobot.com/LCD_KeyPad_Shield_For_Arduino_SKU_DFR0009
9. "Automatic card shuffler machine," YouTube, 2026, video consultado el 4 de marzo de 2026.
10. "Card dealer machine prototype," YouTube, 2026, video consultado el 4 de marzo de 2026.
11. "Automatic card dispenser system," YouTube, 2026, video consultado el 4 de marzo de 2026.
12. "Easy to make - motorized lazy susan," <https://www.youtube.com/watch?v=kJvAgkTQ9qY>, 2020, accedido el 15 de abril de 2026.
13. Arduino, *Arduino Mega 2560 Rev3*, 2024, datasheet oficial. [Online]. Available: <https://docs.arduino.cc/hardware/mega-2560/>
14. Equipo ELCO-Dealer, "Repositorio github del proyecto elco-dealer," 2026, repositorio público del código fuente.
15. —, "Guía instructables del proyecto elco-dealer," 2026, guía pública de montaje y replicación.